

Building Clinical AI Agents

From FHIR Fundamentals to Autonomous Orchestration

```
1  {
2    "resourceType": "Bundle",
3    "total": 1243,
4    "entry": [
5      {
6        "resource": {
7          "resourceType": "Patient",
8          "id": "abc123",
9          "name": [{"given": "Jane"}, {"family": "Doe"}],
10         "birthDate": "1965-04-12",
11         "gender": "female"
12       }
13     ]
14  }
```



PATIENT SUMMARY

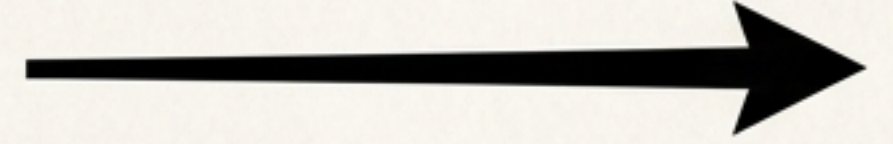


Jane Doe

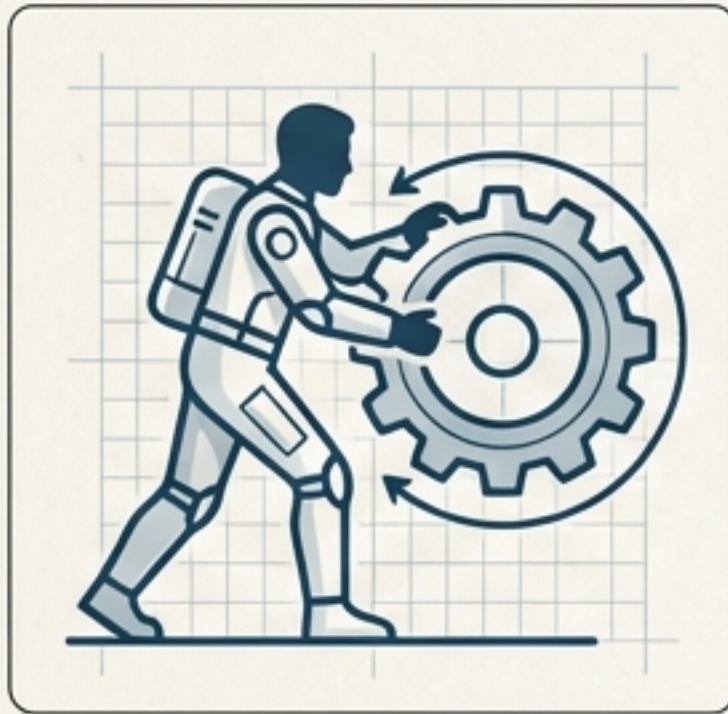
Vital	Value	Unit	Status
Blood Pressure	120/80	mmHg	Normal
Heart Rate	72	bpm	Normal
Temperature	36.8	°C	Normal

Clinical Editorial meets Modern Engineering

Datatipography, data/code use JetBrains Mono

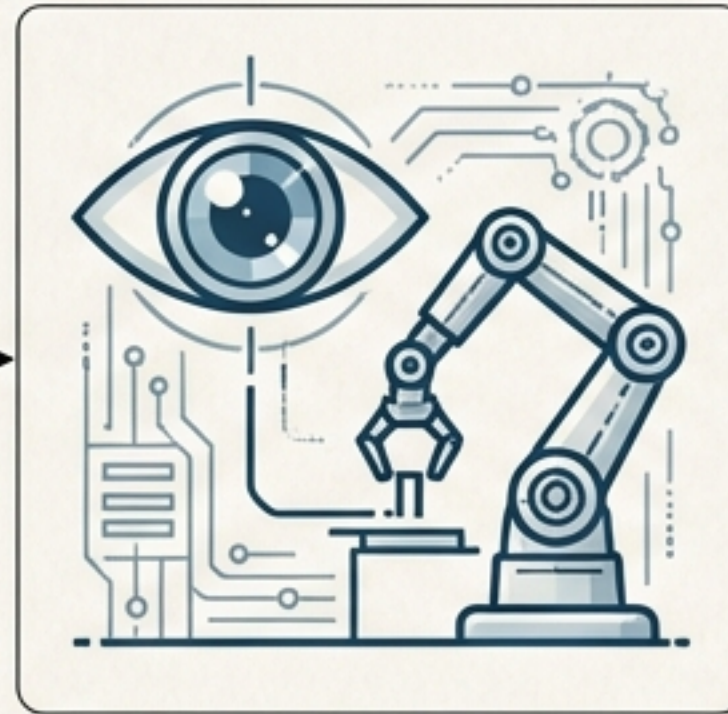


Level 1: The Pilot



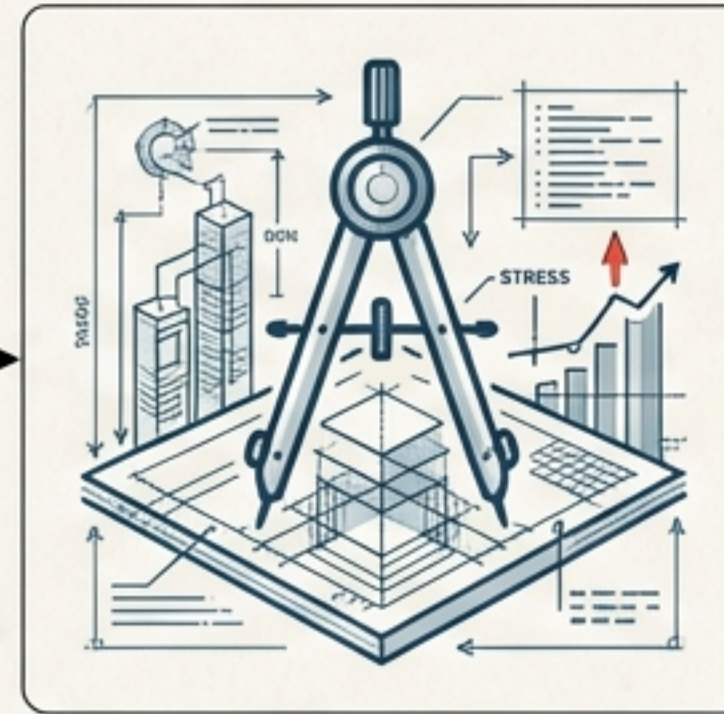
Session 1: Human orchestrates, LLM writes code.

Level 2: The Observer



Session 2: AI Agent orchestrates, Human verifies.

Level 3: The Architect



Session 3: Stress-testing and finding failure modes.

“The manual steps you do today become the tools the agent uses autonomously.”

We move from writing queries to designing to the systems that write them.

The Clinical Logic Challenge

Diagnosis

Type 2 Diabetes

SNOMED: 44054006

(Not ICD-10)



Demographics

Name & DOB

Source: Patient Resource

Metric

HbA1c Level

LOINC: 4548-4

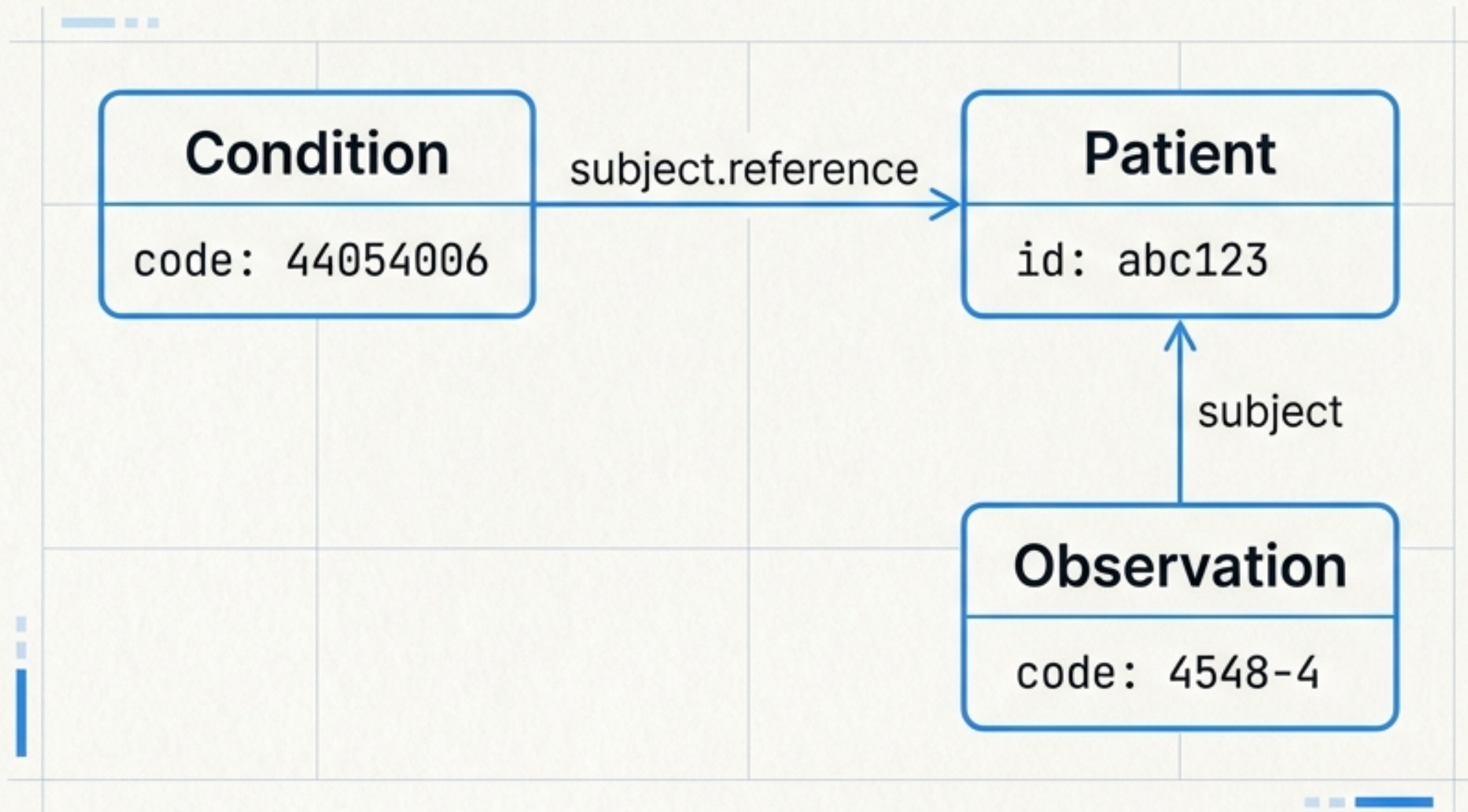
Mission Objective

Mission: Find patients with Type 2 diabetes, retrieve their most recent HbA1c values, and identify those with poor glycemic control.

**Poor Control
Threshold:
> 7.0%**

The Data Layer: FHIR R4

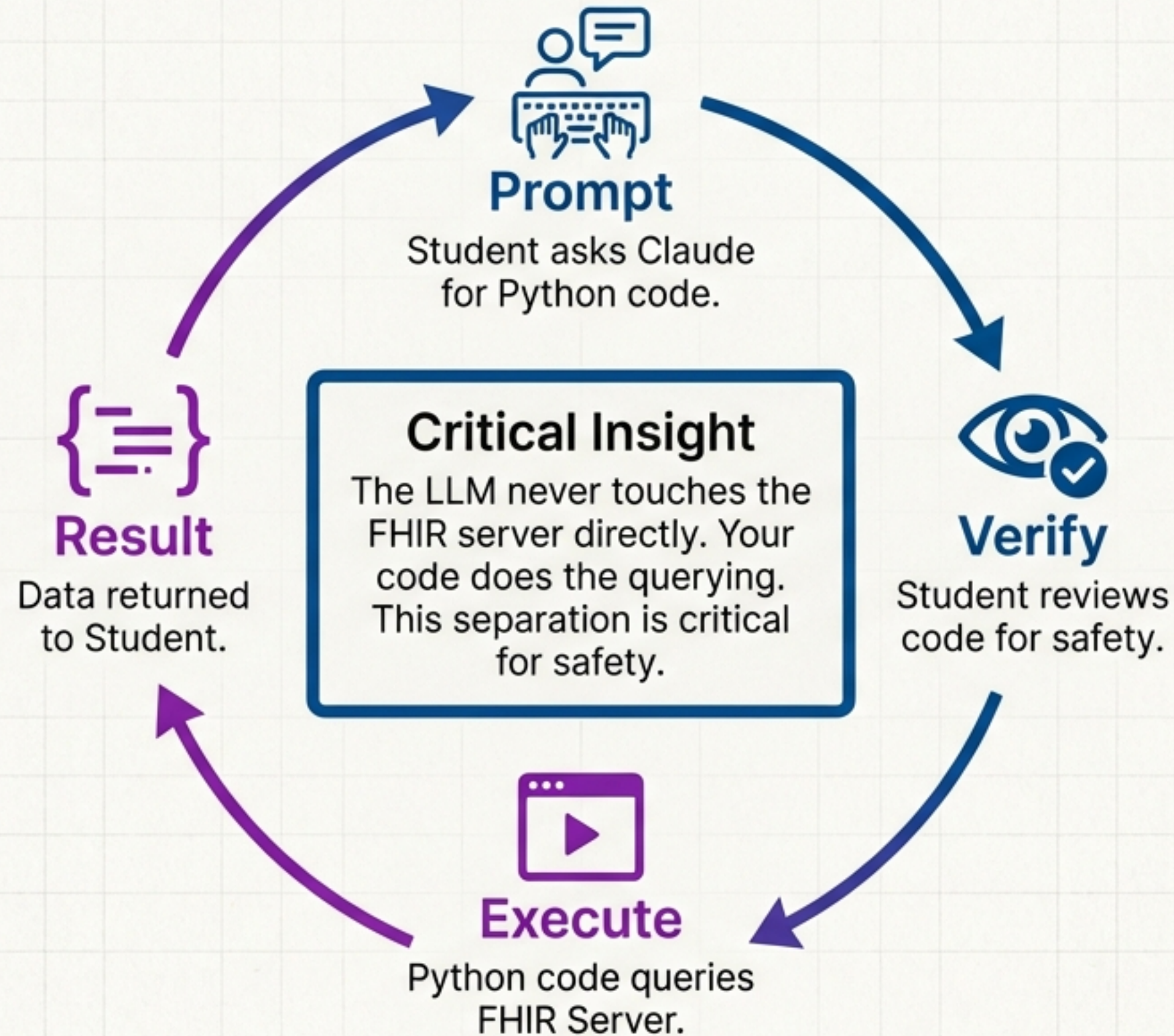
Fast Healthcare Interoperability Resources



Think of it as a RESTful API for healthcare. No single query gives you the full answer. You must follow the chain of references.

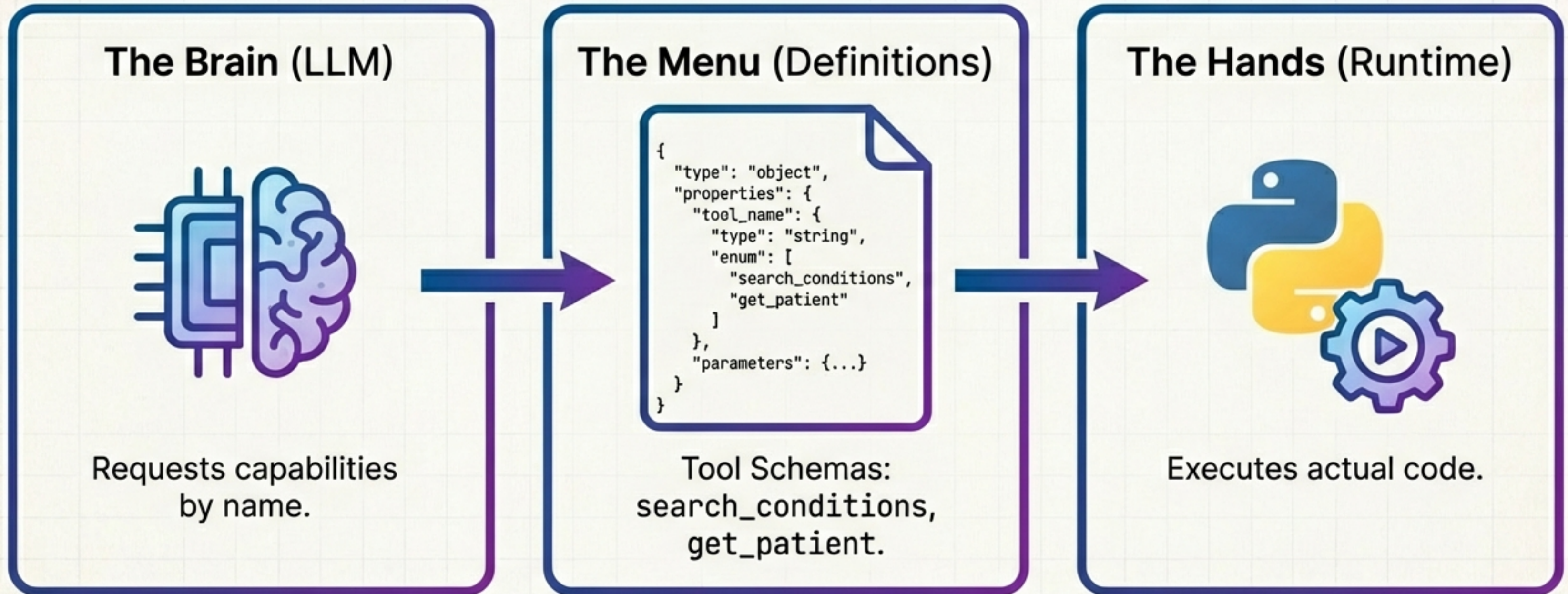
Standard Vocabularies:
SNOMED CT for Conditions,
LOINC for Observations.

Session 1: The Human-in-the-Loop



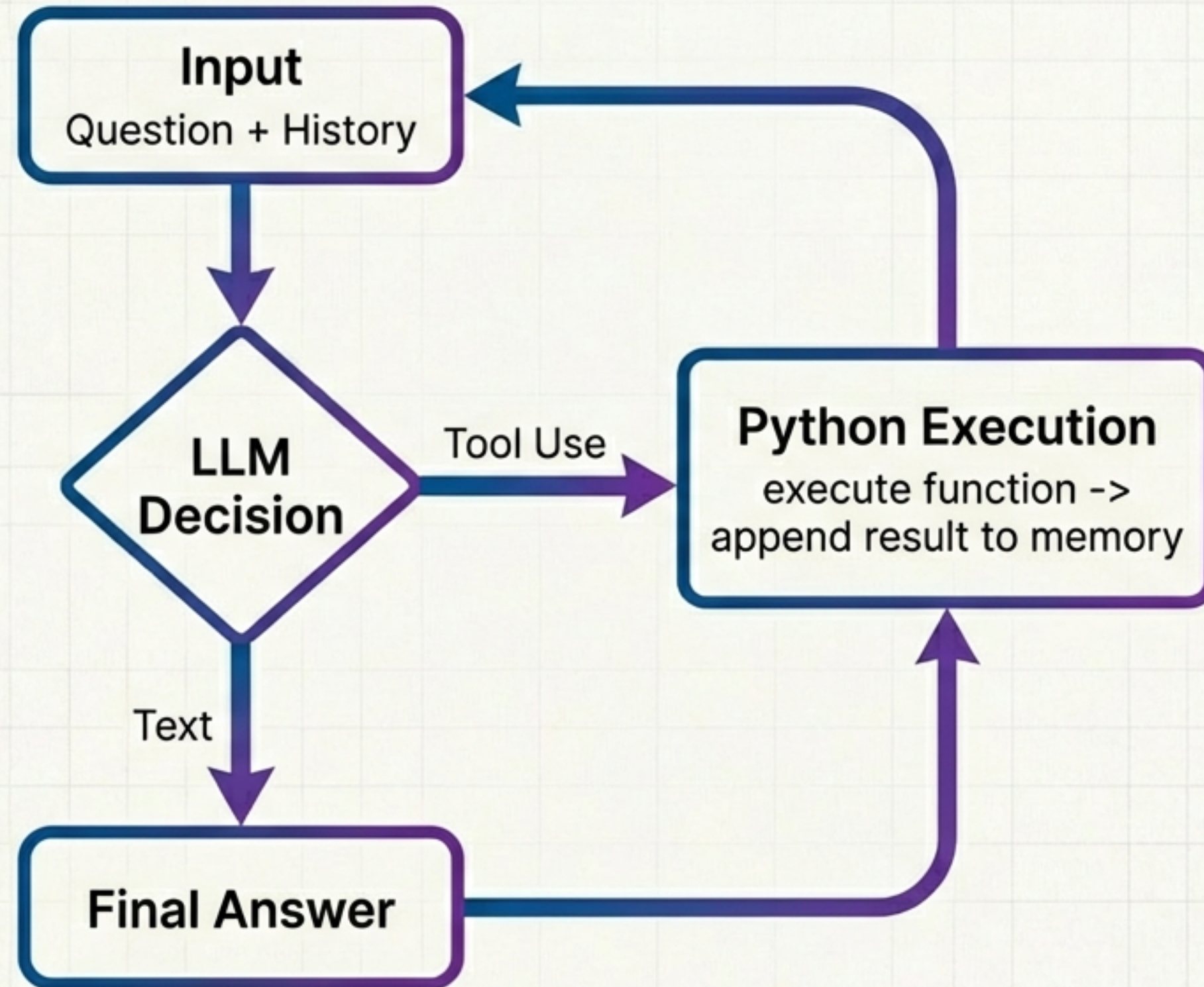
The Paradigm Shift: Tool Use

The Menu and The Kitchen



The LLM doesn't see the code. It sees a menu of capabilities (Schemas) and requests them by name.

Architecture of the Agent Loop



Memory is transient.
Nothing persists
between calls except
the message list.

The Agent's Toolbox

Core Tools - Sessions 1 & 2

```
def search_conditions(code) {  
  return -> SPS eoscaonter:  
  "ool_command": {  
    "properties": {  
      "patient": {  
        get {  
          "search_conditions",  
          "search_conditions(patient_idfn(code))  
        }  
      }  
    }  
  }  
}
```



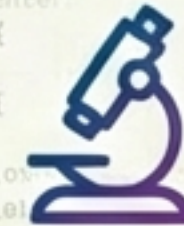
search_conditions(code)

```
def get_patient(patient_id) {  
  return -> SPS eoscaonter:  
  "ool_command": {  
    "properties": {  
      "patient": {  
        get {  
          "search_conditions",  
          "search_conditions(patient_id, loine)  
        }  
      }  
    }  
  }  
}
```



get_patient(patient_id)

```
def search_observations(patient_id, loinc) {  
  return -> SPS eoscaonter:  
  "ool_command": {  
    "properties": {  
      "patient": {  
        get {  
          "search_observations",  
          "search_observations(patient_id, loinc)  
        }  
      }  
    }  
  }  
}
```



search_observations
(patient_id, loinc)

Extended Tools - Session 3

```
public search_medications(patient_id) {  
  return -> the sequence:  
  "tool_session": {  
    "properties": {  
      "patient": {  
        get {  
          "long_search",  
          "search_medications(patient_id)  
        }  
      }  
    }  
  }  
}
```



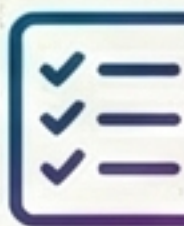
search_medications(patient_id)

```
public search_encounters(patient_id) {  
  return -> the sequence:  
  "tool_session": {  
    "properties": {  
      "patient": {  
        get {  
          "long_search",  
          "search_encounters(patient_id)  
        }  
      }  
    }  
  }  
}
```



search_encounters(patient_id)

```
public search_all_conditions(patient_id) {  
  return -> the sequence:  
  "tool_session": {  
    "properties": {  
      "patient": {  
        get {  
          "long_search",  
          "search_all_conditions(patient_id)  
        }  
      }  
    }  
  }  
}
```



search_all_conditions(patient_id)

Anatomy of a Successful Trace

```
> User: Find diabetic patients and their HbA1c.  
[THOUGHT] I need to find patients with Type 2 Diabetes first.  
[CALL] search_conditions(code='44054006')  
[RESULT] Found 30 conditions. References: Patient/1, Patient/2...  
[THOUGHT] Now I need demographics for these patients.  
[CALL] get_patient(patient_id='1')  
[THOUGHT] Checking lab values for Patient 1.  
[CALL] search_observations(patient_id='1', loinc='4548-4')
```

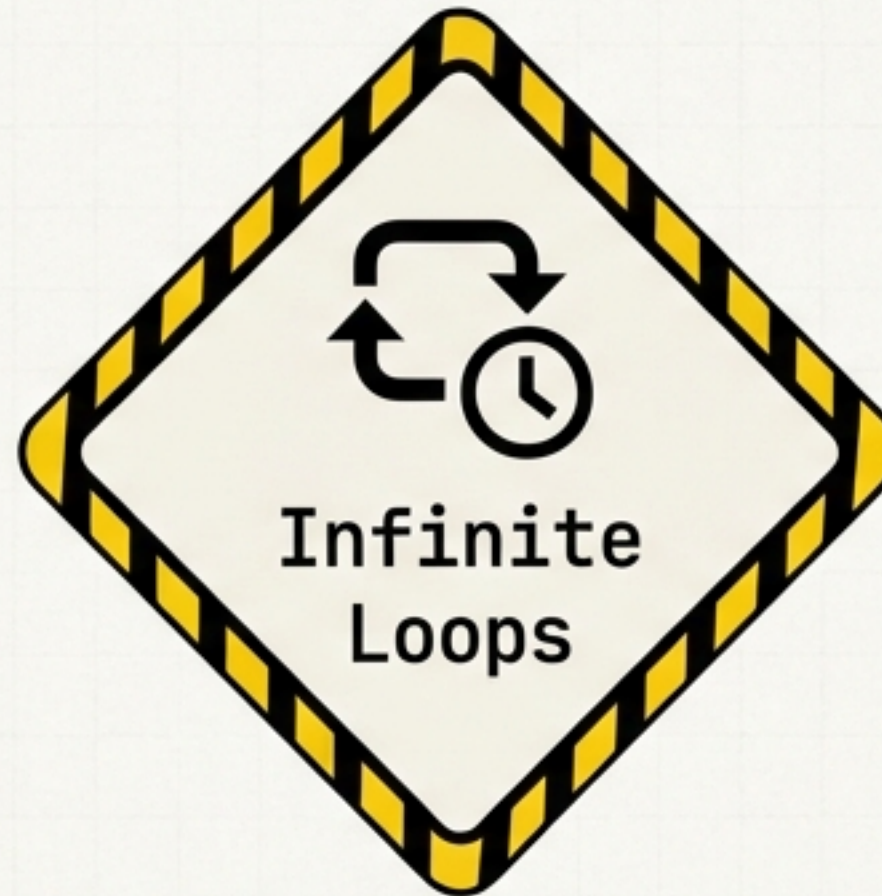
Predictability is key.
The agent follows the
manual clinical
workflow
autonomously.

Session 3: The Quality Assurance Mindset

Finding Failures = Success



Calling functions or SNOMED codes that don't exist.



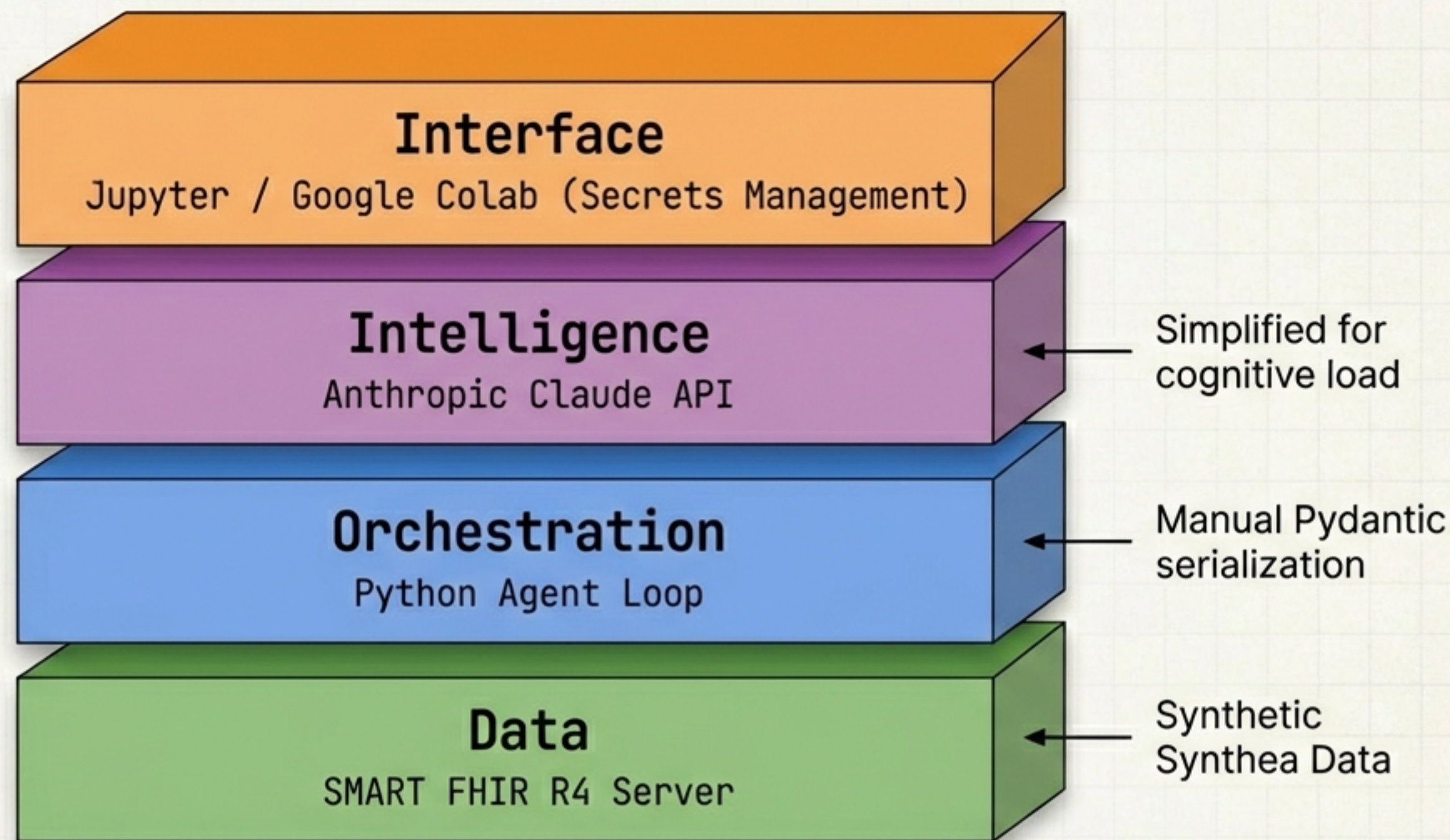
Getting stuck asking for the same patient data repeatedly.



Handling missing Creatinine or Hypertension data gracefully.

The goal of Session 3 is to break the system to understand its limits.

Technical Stack & Constraints



Design Decisions & Trade-offs

Hash Verification 	Removed. JSON logs with timestamps are sufficient for grading.
Annotated Notebooks 	Script-generated for synchronization (create_annotated_notebooks.py).
Single Provider 	Anthropic only. Reduces API key friction compared to dual-provider.

Key Learning Takeaways

Separation of Concerns

Intelligence (LLM) vs. Execution (Code).
The LLM never touches the DB directly.

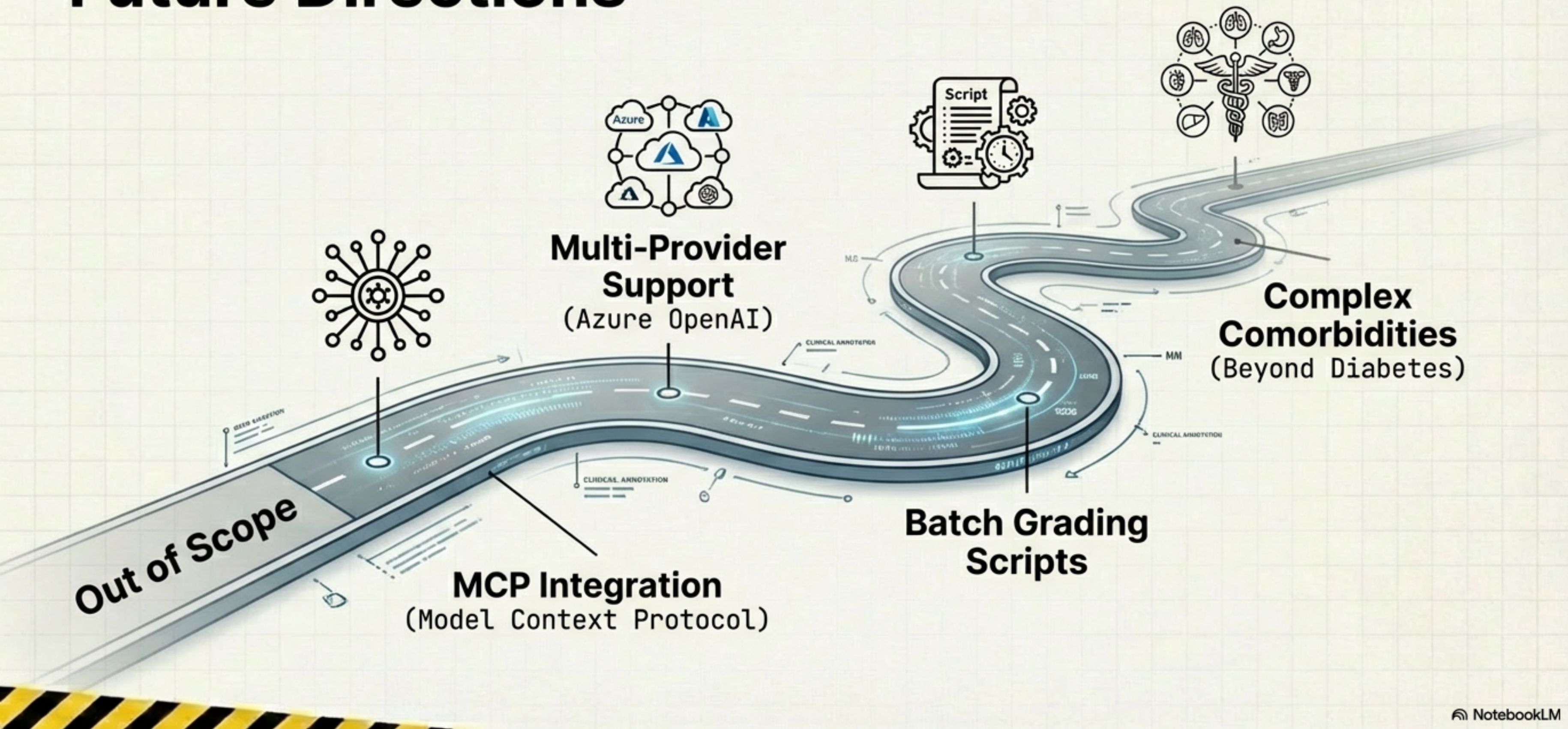
Structured Inputs

Precision matters.
Success depends on specific SNOMED/LOINC codes.

Orchestration

The shift from writing queries manually to defining tools for an agent.

Future Directions



Resources & Critical Codes

LINKS

FHIR Server:

<https://launch.smarthealthit.org/v/r4/fhir>

Interface:

claude.ai

CODES

Type 2 Diabetes:

SNOMED 44054006

HbA1c:

LOINC 4548-4

Poor Control Threshold:

> 7.0%

“ The agent is only as good as its tool definitions and its system prompt. ”